

Unsupervised learning

This is a branch of machine learning that includes problems where we do not have true labels for the data to train with. Instead, the goal is to somehow organize the data into some meaningful clusters or densities. It is usually concerned with trying to extract meaningful insights or information from data rather than trying to predict some outcome based on previously available supervised training data i.e., the algorithms tries to learn inherent latent structures, patterns and relationships from given data without any help or supervision like providing annotations in the form of labeled outputs or outcomes. Unsupervised learning methods are categorized under the following broad areas of machine learning

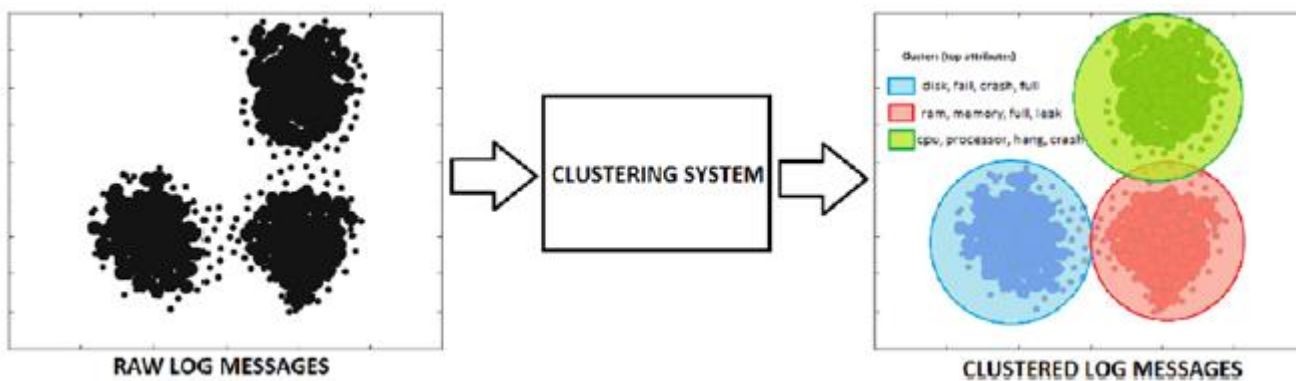
- i). Clustering
- ii). Dimensionality reduction
- iii). Anomaly detection
- iv). Association rule-mining

Clustering

Clustering methods are Machine Learning methods that try to find patterns of similarity and relationships among data samples in a dataset and then cluster these samples into various groups, such that each group or cluster of data samples has some similarity, based on the inherent attributes or features.

The goal is to split up the data in such a way that points within a single cluster are very similar and points in different clusters are different. Clustering algorithms will assign (or predict) a number to each data point, indicating which cluster a particular point belongs to.

The following diagram shows clustering of log messages for typical issues or errors. The first cluster depicts disk issues, the second cluster is about memory issues, and the third cluster is about processor issues.



The following are various types of clustering methods classified under the following major approaches.

- i). Partition based clustering e.g., K-means and K-medoids
 - ii). Hierarchical clustering methods e.g., agglomerative and divisive
 - iii). Density based methods e.g., dbscan and optics.
 - iv). Distribution based clustering methods such as Gaussian mixture models
- i). **Partition based clustering:** This is based on the notion of *similarity*, which can be any measure that can be derived from the attributes of the data points by applying mathematical functions on this attributes (features). Then, you can group data points that are similar to each other in a single group and separate the ones that are different. A partition based clustering model is usually developed using a recursive technique, i.e., start with some arbitrary partitions of data and, based on the similarity measure, keep reassigning data points until you reach a stable stopping criteria. Examples include techniques like K-means, K-medoids, CLARANS etc.

k-Means Clustering

k-means clustering is one of the simplest and most commonly used clustering algorithms. It tries to find cluster centers that are representative of certain regions of the data. The algorithm works as follows:

1. First, you need to determine the number of clusters (k) that you want. If our data points are in n dimensions, the centers of these clusters, or centroids, will also be n-dimensional points. In other words, we will have a total of k n-dimensional points that we will call the centroids. Each cluster center is the mean of the data points that are assigned to it
2. Then assign each of the actual data points to one of these k centroids based on their distances to these centroids. After this step, each data point will be assigned to one of the k clusters.
3. Re-compute each cluster's centroid. So, we end up with k centroids again, but these are now adjusted to reflect how the data points are distributed. Keep repeating steps 2 and 3 until convergence.

The algorithm finishes when the assignment of instances to clusters no longer changes and at that point we have “real” k clusters, with each data point belonging to one of them. Fortunately, with the right package in Python, we do not need to implement all of this from scratch. `sklearn.cluster`, contains implementations for various clustering algorithms, including k-means.

Example: The following is an example of applying synthetic data to KMeans algorithm and then setting the number of clusters to 3. The fit method is used with the data to create the model

```
In[1]:
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
X, y = make_blobs(random_state=1)
# build the clustering model
kmeans = KMeans(n_clusters=3)
kmeans.fit(X)
```

```
Out[1]:
KMeans(n_clusters=3)
```

During the algorithm, each training data point in X is assigned a cluster label. You can find these labels in the `kmeans.labels_` attribute:

```
In[2]:
print("Cluster memberships:\n{}".format(kmeans.labels_))
```

```
Out[2]:
Cluster memberships:
[0 1 1 1 2 2 2 1 0 0 1 1 2 0 2 2 2 0 1 1 2 1 2 0 1 2 2 0 0 2 0 0 2 0 1 2
 1 1 1 2 2 1 0 1 1 2 0 0 0 0 1 2 2 2 0 2 1 1 0 0 1 2 2 1 1 2 0 2 0 1 1 1
 2 0 0 1 2 2 0 1 0 1 1 2 0 0 0 0 1 0 2 0 0 1 1 2 2 0 2 0]
```

You can also assign cluster labels to new points, using the `predict` method. Each new point is assigned to the closest cluster center. Running `predict` on the training set returns the same result as `labels_` as shown below

```
In[3]:
print(kmeans.predict(X))

Out[3]:
[0 1 1 1 2 2 2 1 0 0 1 1 2 0 2 2 2 0 1 1 2 1 2 0 1 2 2 0 0 2 0 0 2 0 1 2
 1 1 1 2 2 1 0 1 1 2 0 0 0 0 1 2 2 2 0 2 1 1 0 0 1 2 2 1 1 2 0 2 0 1 1 1
 2 0 0 1 2 2 0 1 0 1 1 2 0 0 0 0 1 0 2 0 0 1 1 2 2 0 2 0]
```

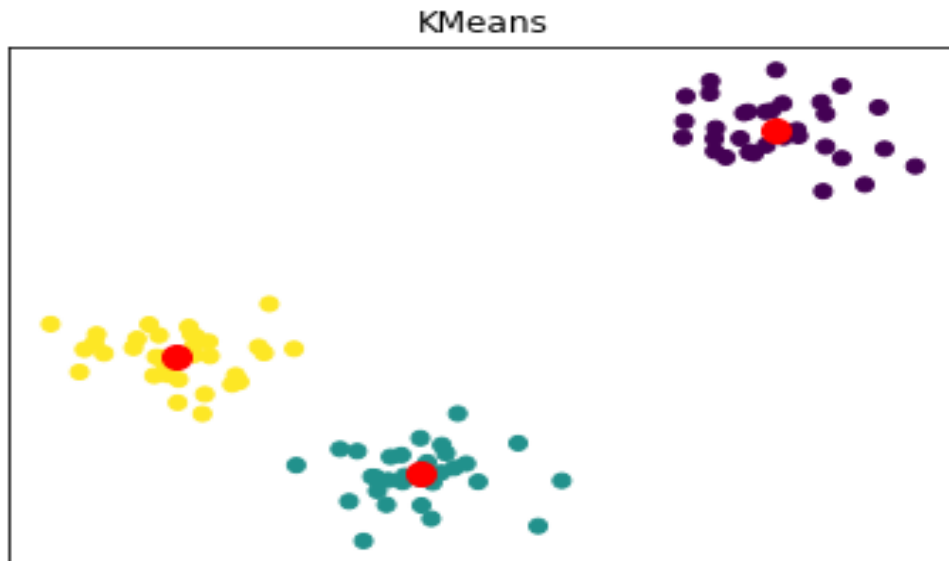
The cluster centers are stored in the `cluster_centers_` attribute

The following is visualization of the results by plotting the data colored based on their labels and cluster centers as determined by the k-means estimator

In[4]:

```
import matplotlib.pyplot as plt
%matplotlib inline
y_kmeans = kmeans.labels_
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans)
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], color='r', s=100)
# Title of the plot
plt.title('KMeans')
# Remove x-axis ticks
plt.xticks(())
# Remove y-axis ticks
plt.yticks(())
```

Out[4]:



- ii). **Hierarchical clustering:** In this paradigm, you can either start with all the data points in one group (divisive clustering) or all the data points in different groups (agglomerative). Based on the starting point you can either keep dividing the big group into smaller groups or clusters based on some accepted similarity criteria or you can keep merging different groups or clusters into bigger ones based on the same criteria. This process is normally stopped when a decided stopping condition is achieved. Similarity criteria could be inter-data point distance in a cluster as compared to other cluster data points. Examples include Ward's minimum variance criterion based agglomerative hierarchical clustering.

Agglomerative Clustering

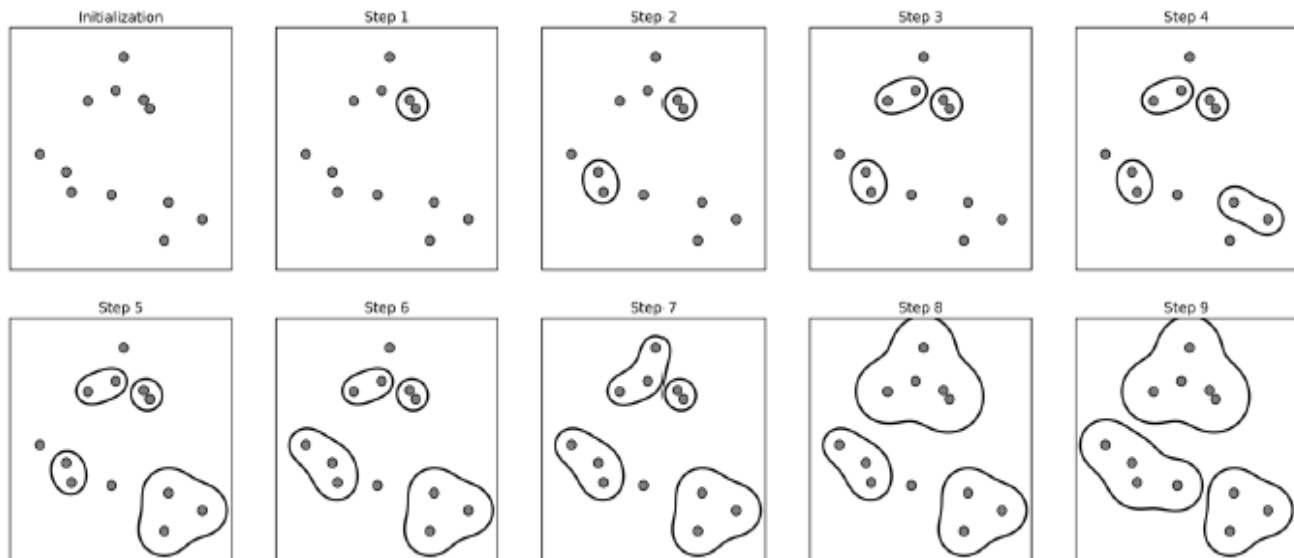
This algorithm starts by declaring each point on its own cluster, and then merges the two most similar clusters until some stopping criterion is satisfied. The stopping criterion implemented in scikit-learn is the number of clusters, so similar clusters are merged until only the specified number of clusters are left. There are several *linkage* criteria that specify how exactly the “most similar cluster” is measured. This measure is always defined between two existing clusters.

The following three choices are implemented in scikit-learn:

- Ward: - The default choice, ward picks the two clusters to merge such that the variance within all clusters increases the least. This often leads to clusters that are relatively equally sized.
- Average: - average linkage merges the two clusters that have the smallest average distance between all their points.
- Complete: - complete linkage (also known as maximum linkage) merges the two clusters that have the smallest maximum distance between their points.

Ward works on most datasets. If the clusters have very dissimilar numbers of members (if one is much bigger than all the others, for example), average or complete might work better.

The following diagram illustrates the progression of agglomerative clustering on a two-dimensional dataset, looking for three clusters



Initially, each point is its own cluster. Then, in each step, the two clusters that are closest are merged. In the first four steps, two single-point clusters are picked and these are joined into two-point clusters. In step 5, one of the two-point clusters is extended to a third point, and so on. In step 9, there are only three clusters remaining.

Example

The following shows how agglomerative clustering performs on the simple three cluster data. Because of the way the algorithm works, agglomerative clustering cannot make predictions for new data points and has no predict method. To build the model and get the cluster memberships on the training set, use the fit_predict method.

In[5] :

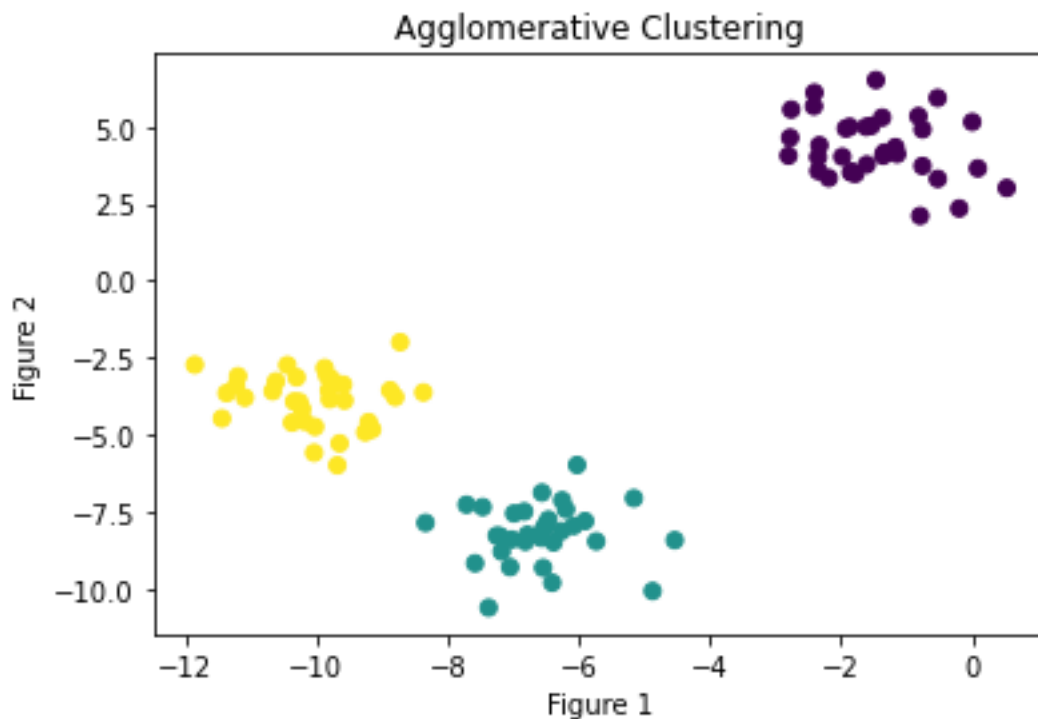
```
from sklearn.cluster import AgglomerativeClustering
X, y = make_blobs(random_state=1)
model = AgglomerativeClustering(n_clusters=3)
model=model.fit(X)
assignment = model.fit_predict(X)
agg.labels_

array([3, 2, 2, 2, 0, 0, 0, 2, 1, 1, 2, 2, 0, 1, 0, 0, 0, 1, 2, 2, 0, 2,
       0, 1, 2, 0, 0, 1, 3, 0, 1, 1, 0, 1, 2, 0, 2, 2, 2, 0, 0, 2, 1, 2,
       2, 0, 1, 3, 1, 1, 2, 0, 0, 0, 1, 0, 2, 2, 1, 1, 2, 0, 0, 2, 2, 0,
```

```
1, 0, 1, 2, 2, 2, 0, 1, 1, 2, 0, 0, 1, 2, 1, 2, 2, 0, 1, 3, 1, 3,
2, 1, 0, 1, 1, 2, 2, 0, 0, 1, 0, 3], dtype=int64)
```

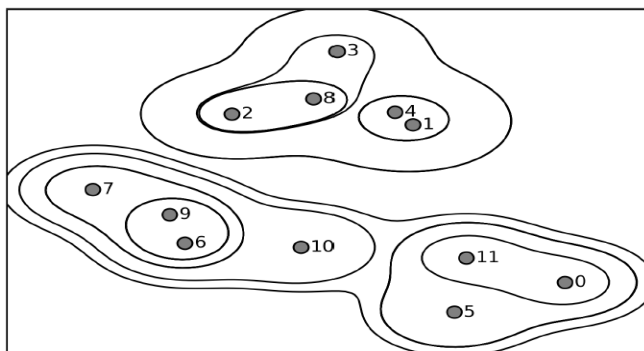
The following is visualization of the results by plotting the data colored based on their labels

```
In[6]:
from matplotlib import pyplot as plt
plt.scatter(X[:, 0], X[:, 1], c=assignment)
# Title of the plot
plt.title('Agglomerative Clustering')
plt.xlabel('Figure 1')
plt.ylabel('Figure 2')
```



Hierarchical clustering and dendrograms

In the following example shows an overlay of all the possible clustering, providing some insight into how each cluster breaks up into smaller clusters:



While this visualization provides a detailed view of the hierarchical clustering, it relies on the two-dimensional nature of the data and cannot be used on datasets that have more than two features. There is another tool to visualize hierarchical clustering, called a *dendrogram*, that can handle multidimensional datasets.

However, scikit-learn does not have the functionality to draw dendrograms. Although, it can be generated easily using SciPy. The SciPy clustering algorithms have a slightly different interface to the scikit-learn clustering algorithms.

SciPy provides a function that takes a data array X and computes a *linkage array*, which encodes hierarchical cluster similarities. The linkage array is fed into the scipy dendrogram function to plot the dendrogram as shown below

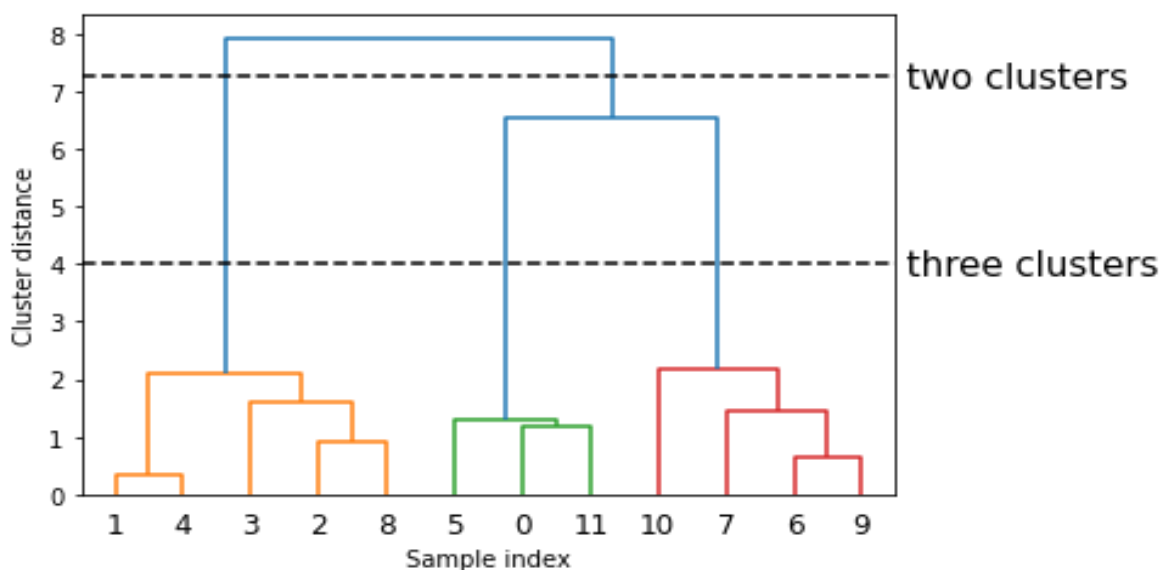
```
In[7]:
from sklearn.datasets import make_blobs
# Import the dendrogram function and the ward clustering function
from scipy.cluster.hierarchy import dendrogram, ward
X, y = make_blobs(random_state=0, n_samples=12)
# The SciPy ward function returns an array that specifies the distances
# bridged when performing agglomerative clustering
linkage_array = ward(X)
```

Plot the dendrogram for the linkage_array containing the distances between clusters

```
In[8]:
dendrogram(linkage_array)
```

Mark the cuts in the tree that signify two or three clusters

```
In[9]:
from matplotlib import pyplot as plt
ax = plt.gca()
bounds = ax.get_xbound()
ax.plot(bounds, [7.25, 7.25], '--', c='k')
ax.plot(bounds, [4, 4], '--', c='k')
ax.text(bounds[1], 7.25, ' two clusters', va='center',
fontdict={'size': 15})
ax.text(bounds[1], 4, ' three clusters', va='center',
fontdict={'size': 15})
plt.xlabel("Sample index")
plt.ylabel("Cluster distance")
```



The length of each branch shows how far apart the merged clusters are. The longest branches in this dendrogram are the three lines that are marked by the dashed line labeled “three clusters.” i.e. these are the longest branches indicates that going from three to two clusters meant merging some very far-apart points. This is also seen at the top of the chart, where merging the two remaining clusters into a single cluster again bridges a relatively large distance.

- iii). **Density based clustering:** Both the clustering models mentioned previously are quite dependent on the notion of distance. This can become a problem when we have arbitrary shaped clusters in the data. This limitation can be addressed by doing away with the concept of a distance metric-based clustering. We can define a notion of “density” of data and use it to develop clusters. The cluster development methodology then changes from finding points in the vicinity of some points to finding areas where we have some data points. This approach is not as straightforward to interpret as the distance metric approach but it leads to clusters that necessarily need not be spherical. This is very desirable trait as it is unlikely that all the clusters of interest will be spherical in shape. Examples include DBSCAN and OPTICS.

DBSCAN (Density Based Spatial Clustering of Applications with Noise)

This algorithm work by identifying points that are in “crowded” regions of the feature space, where many data points are close together (*dense regions*) and it can be used capture clusters of *complex shapes*. Points that are within a dense region are called **core points**.

There are two parameters in DBSCAN: min_samples and eps. These represent the minimum data points within a distance of eps to a given data point that form a core sample.

If there are less than min_samples points within distance eps of the starting point, this point is labeled as **noise**, meaning that it doesn’t belong to the cluster. These points will be assigned a new cluster label. The cluster is grown by repeatedly visiting neighbours that have not been visited until there are no more core samples within distance eps of the cluster. Then procedure is repeated.

In the end, there are three kinds of points: **core points**, points that are within distance eps of core points (called **boundary points**), and **noise**. When the DBSCAN algorithm is run on a particular dataset multiple times, the clustering of the core points is always the same, and the same points will always be labeled as noise hence setting a random seed is not required.

Example

The following is an example of DBSCAN based on the iris dataset

```
In[10]:
from sklearn.cluster import DBSCAN
from sklearn.datasets import load_iris
iris_dataset=load_iris()
X=iris_dataset.data
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
X=sc.fit_transform(X)
model=DBSCAN().fit(X)
assignment=model.fit_predict(X)
print("Cluster membership: ", assignment)

out [10]
Cluster membership: [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 -1 -1  0  0  0  0
 0  0  0  0  0  0  0  0  0  0  0  0  0 -1 -1  0  0  0  0  0  0  0  0  0
 0  1  1  1  1  1  1 -1 -1  1 -1 -1  1 -1  1  1  1  1 -1  1  1  1 -1  1  1  1
 1  1  1  1  1  1  1  1 -1  1 -1  1  1  1  1 -1  1  1  1 -1  1 -1  1  1  1 -1
 1 -1 -1 -1  1  1  1  1 -1  1  1 -1 -1 -1  1  1 -1  1  1  1 -1 -1 -1  1  1
 1 -1 -1  1  1  1  1  1  1  1  1  1  1 -1  1]
```

Notice some data points were assigned the label -1, which stands for noise. This is a consequence of the default parameter settings for `eps` and `min_samples`, which are not tuned for small datasets. Changing these parameters improves the cluster detection.

The following is output of model visualization

```
In[11]:  
from matplotlib import pyplot as plt  
plt.scatter(X[:,0], X[:,1], c=assignment, cmap='rainbow')  
plt.show()
```

out [11]

